

Application Layer

FILE TRANSFER PROTOCOL (FTP)

- ▶ File Transfer Protocol (FTP) is the standard TCP/IP protocol used for transferring files from one host to another.
- ▶ FTP solves problems caused by different file naming conventions, data representations, and directory structures.
- ▶ It is preferred over HTTP for transferring large files and files of different formats.

FTP Architecture

- ▶ FTP uses a client-server model with two separate TCP connections.

Client Components

User Interface

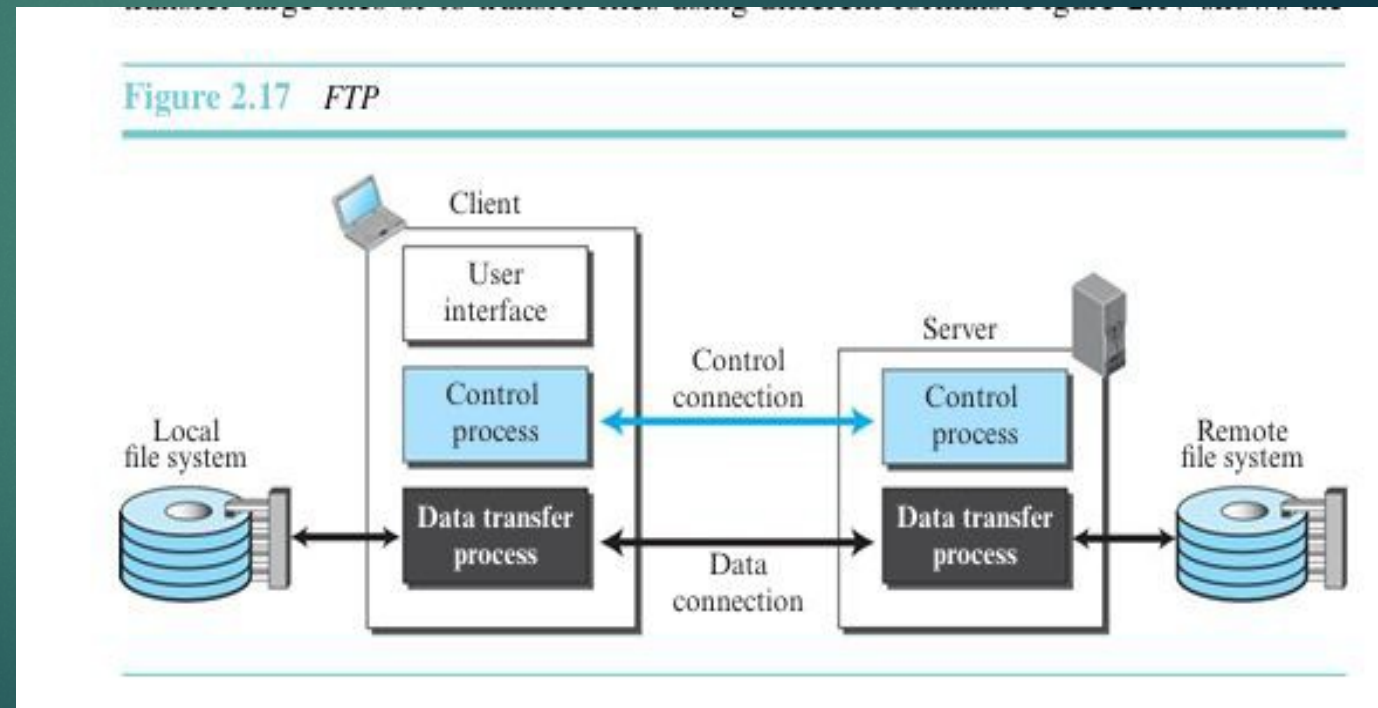
Client Control Process

Client Data Transfer Process

Server Components

Server Control Process

Server Data Transfer Process



FTP Architecture

Connections Used

- ▶ Control Connection – Transfers commands and responses.

Data Connection – Transfers actual files.

- ▶ Advantage: Separation of control information and data transfer makes FTP more efficient.

Lifetimes of FTP Connections

- ▶ Control Connection

Remains open throughout the FTP session.
Used for sending commands and receiving responses.

- ▶ Data Connection

Opened only when a file transfer is required.
Closed after the transfer is completed.
Can be opened and closed multiple times during a single FTP session.

Lifetimes of FTP Connections

- ▶ FTP Port Numbers

Port 21 – Control Connection

Port 20 – Data Connection

- ▶ Control Connection

Uses NVT ASCII character set.

Communication takes place through commands and responses.

Each command or response ends with CRLF (Carriage Return and Line Feed).

FTP Commands

Common FTP Commands

Command	Function
USER	Sends username
PASS	Sends password
LIST	Lists directory contents
RETR	Retrieves a file
STOR	Stores a file
DELE	Deletes a file
QUIT	Ends FTP session
PORT	Specifies client port for data transfer

Establishment of Data Connection

- ▶ The client performs a passive open using an ephemeral port. The client sends the port number to the server using the PORT command.
The server performs an active open using port 20 and the client's ephemeral port.
- ▶ Communication over Data Connection
- ▶ Before file transfer, FTP defines:
- ▶ Data Structure
 - File Type
 - Transmission Mode

Data Structure

- ▶ File Structure (Default) – Continuous stream of bytes.
Record Structure – File divided into records.
Page Structure – File divided into pages with page numbers.

File Types

- ▶ ASCII File

- EBCDIC File

- Image (Binary) File

Transmission Modes

- ▶ Stream Mode (Default) – Continuous stream of bytes.

Block Mode – Data transferred in blocks with headers.

Compressed Mode – Data transferred in compressed form.

FTP Error Codes

FTP Error Codes

Code	Meaning
425	Cannot open data connection
450	File action not taken; file not available
452	Insufficient storage
500	Syntax error; unrecognized command
501	Syntax error in parameters or arguments
530	User not logged in

File Transfer Operations

- ▶ FTP supports three basic operations:

Retrieve (Server → Client)

Transfers a file from the server to the client.

Store (Client → Server)

Transfers a file from the client to the server.

Directory Listing (Server → Client)

Sends directory information from the server to the client.

Electronic Mail (E-Mail)

- ▶ Electronic Mail (E-Mail) is an application-layer service that allows users to **exchange messages** over a computer network or the Internet.
- ▶ Unlike applications such as HTTP and FTP, where a server continuously waits for requests from clients and provides an immediate response, e-mail works differently.
- ▶ E-mail is considered a one-way transaction, meaning that when one user sends a message to another, a response is not mandatory.
- ▶ The receiver may choose to reply or may not respond at all. If a reply is sent, it is treated as a separate one-way transaction.

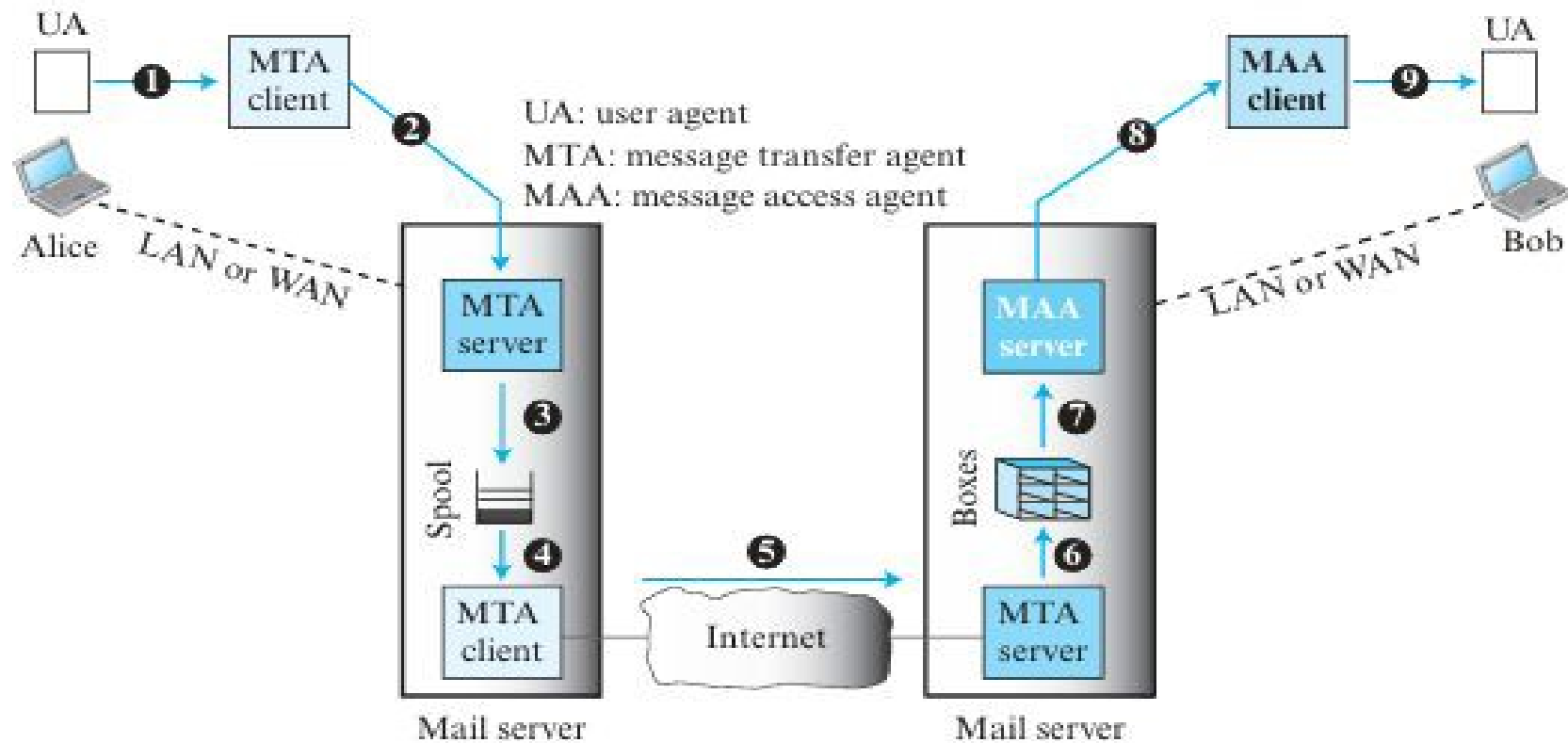
Electronic Mail (E-Mail)

- ▶ In an e-mail system, it is neither practical nor logical for every user to run a server program continuously and wait for incoming messages. Users may switch off their computers or disconnect from the network when they are not using them.
- ▶ To overcome this limitation, **e-mail communication relies on intermediate mail servers.**
- ▶ These servers store, forward, and deliver messages between users. The sender's e-mail client sends the message to a mail server, which then transfers it to the recipient's mail server.
- ▶ When the recipient becomes available, they can access and retrieve the message using their e-mail client.

Electronic Mail (E-Mail)

- ▶ Thus, e-mail implements the **client-server model in an indirect manner.**
- ▶ Users run only client programs whenever they need to send or receive messages, while **dedicated mail servers handle the storage and delivery of e-mails.**
- ▶ This approach allows reliable communication even when the sender and receiver are not online at the same time.

Architecture of E-Mail



Architecture of E-Mail

- ▶ The architecture of an e-mail system consists of users, mail servers, mailboxes, queues, and different software agents that work together to send and receive messages.
- ▶ In a typical e-mail scenario, the sender (Alice) and the receiver (Bob) are connected to their respective mail servers through a LAN or WAN.
- ▶ Each user has a mailbox on the mail server where received messages are stored, and a queue (spool) where outgoing messages wait before being sent.
- ▶ When Alice wants to send an e-mail, she uses a User Agent (UA) to compose and submit the message to her mail server.

Architecture of E-Mail

- ▶ The message is then placed in the queue and transferred across the Internet using a Mail Transfer Agent (MTA). The MTA follows the client-server model, where the client transfers the message to the destination mail server, and the server runs continuously to accept incoming messages.
- ▶ Once the message reaches Bob's mail server, it is stored in Bob's mailbox. To read the message, Bob uses a Message Access Agent (MAA), which retrieves the message from the mail server.
- ▶ Unlike the MTA, which is a push protocol that pushes messages to the destination server, the MAA is a pull protocol that allows users to retrieve messages when needed.
- ▶ This architecture eliminates the need for users to keep their computers running continuously and enables reliable e-mail communication even when the sender and receiver are not online at the same time.

User Agent (UA)

- ▶ A User Agent (UA) is the first component of an electronic mail system that provides services to users for creating, sending, receiving, reading, replying to, and forwarding e-mail messages. It is a software program that acts as an interface between the user and the e-mail system and also manages local mailboxes on the user's computer.
- ▶ User agents are of two types: command-driven and GUI-based. Command-driven user agents were commonly used in the early days of e-mail and require users to enter simple keyboard commands to perform tasks. Examples include mail, pine, and elm.
- ▶ Modern e-mail systems use GUI-based user agents, which provide graphical elements such as icons, menus, buttons, and windows, making e-mail operations easier and more user-friendly.
- ▶ Examples include Microsoft Outlook and Eudora.

Sending Mail

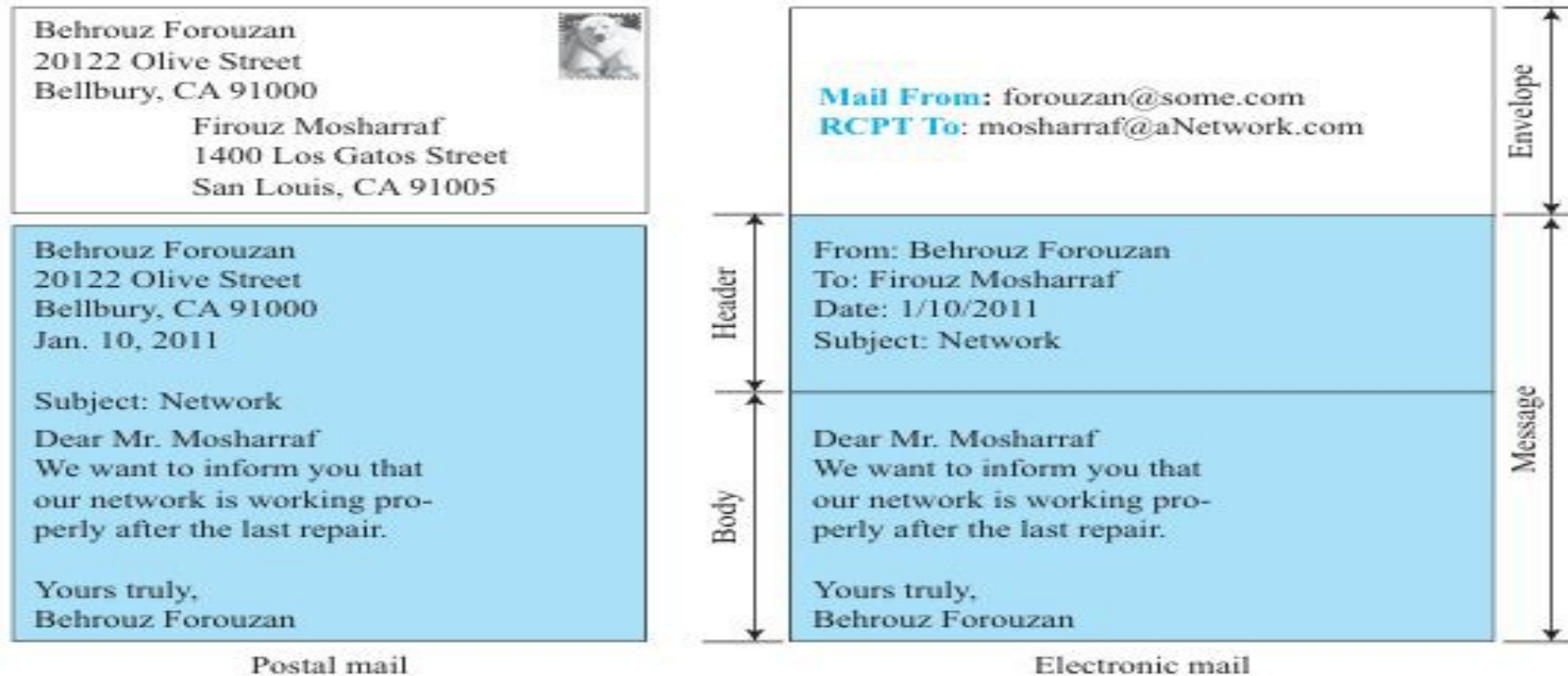
- ▶ To send an e-mail, the user creates the message using the User Agent.
- ▶ An e-mail is similar to postal mail and consists of an envelope and a message.
- ▶ The envelope contains information such as the sender's address, receiver's address, and other delivery details.
- ▶ The message itself contains two parts: a header and a body. The header includes information such as the sender, receiver, subject, and other control details, while the body contains the actual content of the message.
- ▶ Once the message is prepared, the User Agent sends it to the mail server for delivery.

Receiving Mail

- ▶ When receiving e-mail, the User Agent is activated either by the user or automatically through a timer.
- ▶ If new mail is available in the mailbox, the User Agent notifies the user.
- ▶ It then displays a list of received messages, showing summary information such as the sender's e-mail address, subject, and date or time of receipt.
- ▶ The user can select any message from the list and view its complete contents on the screen.
- ▶ Thus, the User Agent simplifies the process of accessing and managing received e-mails.

Format of an e-mail

Figure 2.20 *Format of an e-mail*

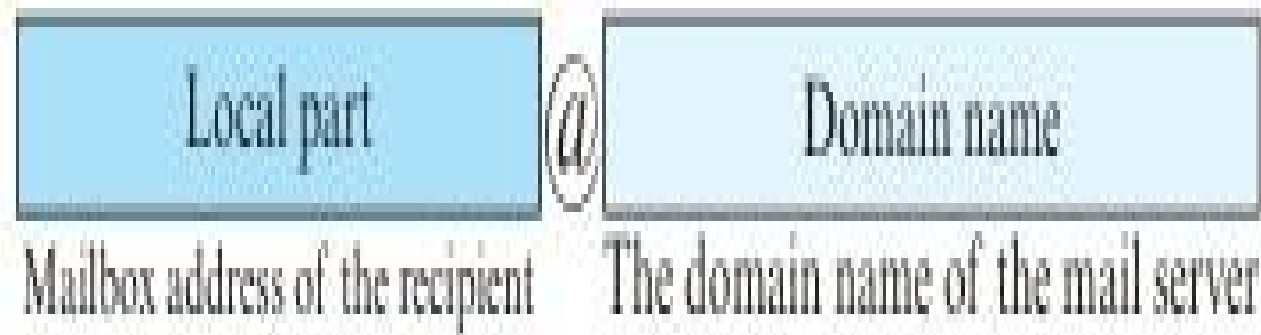


E-Mail Addresses

- ▶ To ensure proper delivery of e-mail messages, an e-mail system uses a unique addressing scheme. In the Internet, an e-mail address consists of **two parts separated by the @ (at) symbol: the local part and the domain name.**
- ▶ **The local part identifies the user's mailbox, which is a special file where all incoming messages for that user are stored until they are retrieved.**
- ▶ The domain name identifies the mail server or mail exchanger responsible for handling the user's e-mails. The domain name may be obtained from the Domain Name System (DNS) or may represent the name of an organization.
- ▶ This addressing system ensures that every e-mail message reaches the correct destination efficiently and accurately.

Email address

Figure 2.21 *E-mail address*



Mailing List (Group List)

- ▶ Electronic mail also supports mailing lists, also known as group lists, which allow a single name or alias to represent multiple e-mail addresses.
- ▶ Instead of sending the same message individually to several recipients, a user can send it to the mailing list alias.
- ▶ The e-mail system then checks the **alias database** and automatically expands the alias into all the e-mail addresses associated with it.
- ▶ Separate copies of the message are created for each member of the group and forwarded to the Mail Transfer Agent (MTA) for delivery.
- ▶ Mailing lists simplify communication with groups and make it easier to send information to multiple recipients simultaneously.

Message Transfer Agent (MTA) – SMTP

- ▶ The Message Transfer Agent (MTA) is responsible for transferring e-mail messages from the sender to the receiver through mail servers.

In an e-mail system, **three client-server interactions** are involved: **the first between the sender and the sender's mail server, the second between the sender's mail server and the receiver's mail server, and the third between the receiver and the receiver's mail server.**

Message Transfer Agent (MTA) – SMTP

- ▶ The first two interactions use a Message Transfer Agent (MTA), while the third uses a Message Access Agent (MAA).
- ▶ The standard Internet protocol used by the MTA is the Simple Mail Transfer Protocol (SMTP). SMTP is used twice during e-mail transmission: first, to transfer the message from the sender's user agent to the sender's mail server, and second, to transfer the message between the sender's and receiver's mail servers.
- ▶ A different protocol, such as POP or IMAP, is used later by the receiver to access the message.

SMTP operates using a command-response mechanism.

- ▶ The MTA client sends commands to the MTA server, and the server replies with appropriate responses. Each command and response is terminated by a two-character end-of-line sequence consisting of a carriage return (CR) and a line feed (LF).
- ▶ Common SMTP commands include HELO (to identify the client), MAIL FROM (to specify the sender's address), RCPT TO (to specify the recipient's address), DATA (to indicate the start of the message content), and QUIT (to terminate the session).
- ▶ Responses are sent from the server to the client in the form of three-digit status codes, often accompanied by textual information that describes the result of the command. Thus, SMTP provides a reliable and standardized method for transferring e-mail messages across the Internet.

Mail Transfer Phases in SMTP

- ▶ The transfer of an e-mail message using Simple Mail Transfer Protocol (SMTP) occurs in three phases: **Connection Establishment, Message Transfer, and Connection Termination**.
- ▶ During the Connection Establishment phase, the SMTP client first creates a TCP connection with the SMTP server using **port 25**. The server then sends **a 220 (Service Ready) response** to indicate that it is ready to receive mail.
- ▶ If the service is unavailable, the server sends a **421 response**. Next, the **client sends a HELO command** along with its domain name to identify itself to the server.
- ▶ The server acknowledges this request by sending a **250 (Request Completed)** response or another appropriate code.

Mail Transfer Phases in SMTP

- ▶ Once the connection is established, the Message Transfer phase begins.
- ▶ The client sends a MAIL FROM command containing the sender's e-mail address, and the server responds with a 250 code if the command is accepted.
- ▶ The client then sends a RCPT TO command specifying the recipient's e-mail address, and the server again responds appropriately. If there are multiple recipients, this step is repeated for each recipient. After specifying the recipients, the client sends the DATA command to indicate the start of the message content.
- ▶ The server responds with 354 (Start Mail Input), allowing the client to transmit the message.

Mail Transfer Phases in SMTP

- ▶ The actual message is then sent line by line, with each **line ending in a carriage return and line feed (CRLF)**. The end of the message is indicated by a line containing only a **single period (.)**. After receiving the complete message, the server sends a **250 (OK)** response to confirm successful receipt.
- ▶ Finally, in the **Connection Termination phase**, the client ends the **SMTP session by sending the QUIT command**. The server responds with a **221 code** to indicate that the connection is being closed. The TCP connection is then terminated. These three phases ensure reliable and orderly transfer of e-mail messages between mail servers on the Internet.

Domain Name System (DNS)

- ▶ The Domain Name System (DNS) is an important client-server application in the TCP/IP protocol suite that helps other Internet applications by translating **human-readable domain names into IP addresses**.
- ▶ Although computers and networking devices communicate using numerical IP addresses, people find it easier to remember names such as google.com or filesource.com.
- ▶ Therefore, DNS acts as **a distributed directory service** that maps domain names to their corresponding IP addresses. Similar to a **telephone directory** that maps people's names to telephone numbers, DNS maps host names to network addresses. Since the Internet contains billions of devices, maintaining a single centralized directory would be impractical and unreliable.

Domain Name System (DNS)

- ▶ Instead, DNS distributes its database across many DNS servers located around the world.
- ▶ This distributed approach improves efficiency, scalability, and reliability because users can access nearby DNS servers, and the failure of one server does not affect the entire system.

Domain Name System (DNS)

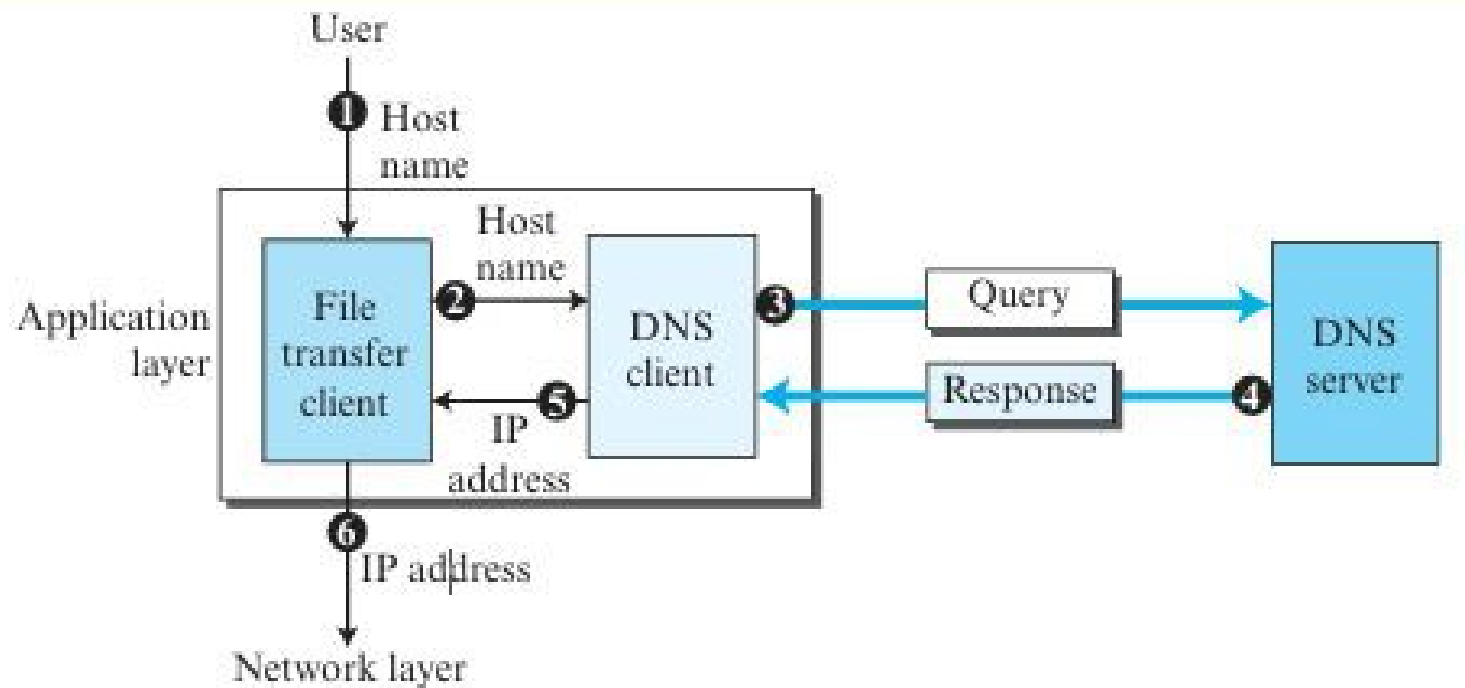
- ▶ When a user wants to access a remote service such as a file transfer server, the user typically knows only the server's domain name and not its IP address.
- ▶ The process begins when the user enters the host name into a client application such as an FTP client.
- ▶ The client passes the host name to the DNS client, which sends a query to a DNS server whose address is already known to the computer. The DNS server searches its database and returns the corresponding IP address of the requested host.
- ▶ The DNS client then provides this IP address to the application client, which uses it to establish a connection with the remote server. Thus, DNS serves as a vital Internet service that enables users to access resources using easy-to-remember names while allowing the TCP/IP protocols to operate using IP addresses.

DNS and Multiple Connections

- ▶ Before a client can connect to a file transfer server, the server's domain name must first be translated into an IP address using DNS.
- ▶ Therefore, at least two connections are required: the first between the DNS client and DNS server for name resolution, and the second between the file transfer client and file transfer server for the actual data transfer.
- ▶ In some cases, DNS may require additional connections to other DNS servers to find the correct IP address.

Domain Name System (DNS)

Figure 2.35 *Purpose of DNS*



Name Space

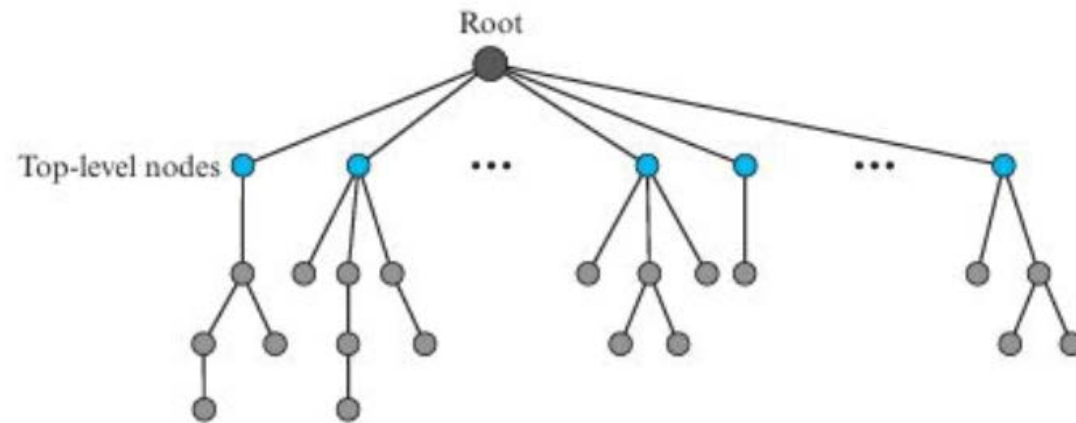
- ▶ A name space is a system used to assign unique names to devices on the Internet so that each name corresponds to a unique IP address.
- ▶ Name spaces can be organized as flat or hierarchical. In a flat name space, each name is simply a sequence of characters without any structure, requiring central control to avoid duplication.
- ▶ This approach is not suitable for large networks like the Internet. A hierarchical name space, on the other hand, consists of multiple levels, allowing the responsibility of assigning names to be distributed among different organizations. This makes it scalable and ensures that names remain unique. For example, two organizations can use the same host name, such as caesar, but their complete domain names (caesar.first.com and caesar.second.com) remain unique.

Domain Name Space

- ▶ To implement a hierarchical naming system, the Internet uses a **Domain Name Space**, which is organized as an **inverted tree structure with the root at the top**.
- ▶ Each node in the tree has a label, which can contain up to **63 characters**. The labels of sibling nodes must be different to ensure uniqueness. Every node also has a domain name, which is formed by a sequence of labels separated by dots (.).
- ▶ **Domain names are read from the node toward the root**. A complete domain name, called a **fully qualified domain name (FQDN)**, ends with the root label, represented by a trailing dot. This hierarchical structure allows DNS to efficiently organize and manage millions of domain names across the Internet.

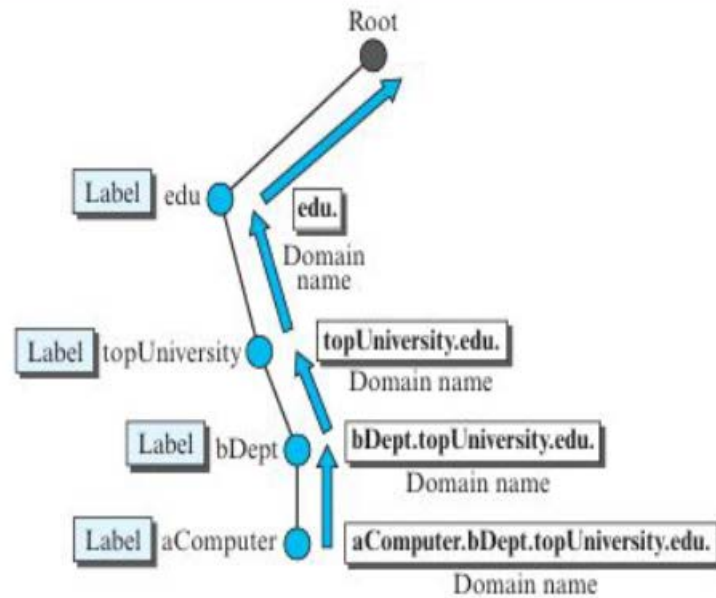
DNS

Figure 2.36 Domain name space



Domain name and labels

Figure 2.37 Domain names and labels

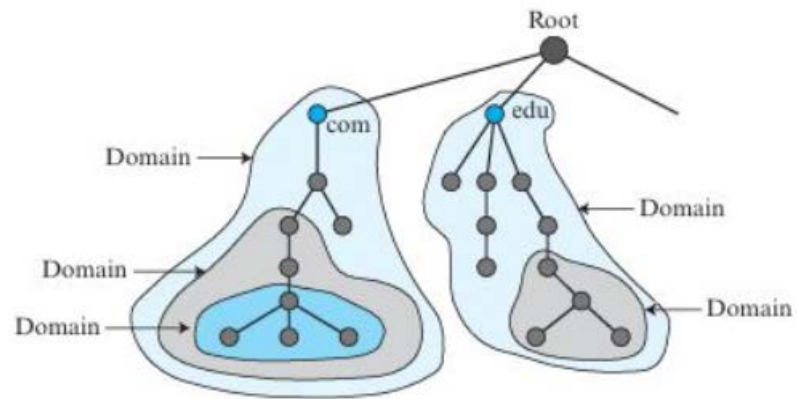


Distribution of Name Space

- ▶ The DNS name space is **distributed among many DNS servers** because storing all domain information on a single server would be inefficient and unreliable.
- ▶ To manage the large amount of data, the domain name space is divided into **domains and subdomains**, and different DNS servers are assigned responsibility for them. This creates a hierarchy of name servers, where each server is authoritative for a particular domain or subdomain.

Domains

Figure 2.38 Domains

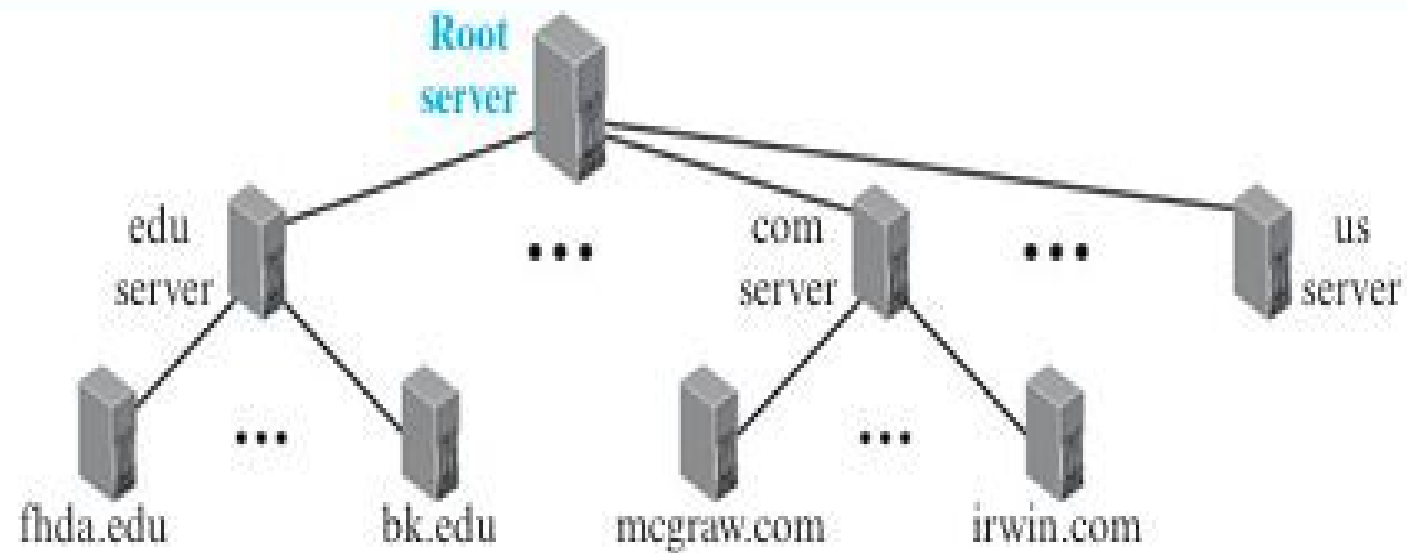


Zone

- ▶ A zone is the portion of the DNS name space for which a DNS server is responsible. If a domain is not divided into subdomains, the domain and zone are the same.
- ▶ However, when a domain is divided and authority is delegated to lower-level servers, the original server retains only part of the information while the detailed information is maintained by the servers responsible for the subdomains. This distributed approach improves the efficiency, scalability, and reliability of the DNS system.

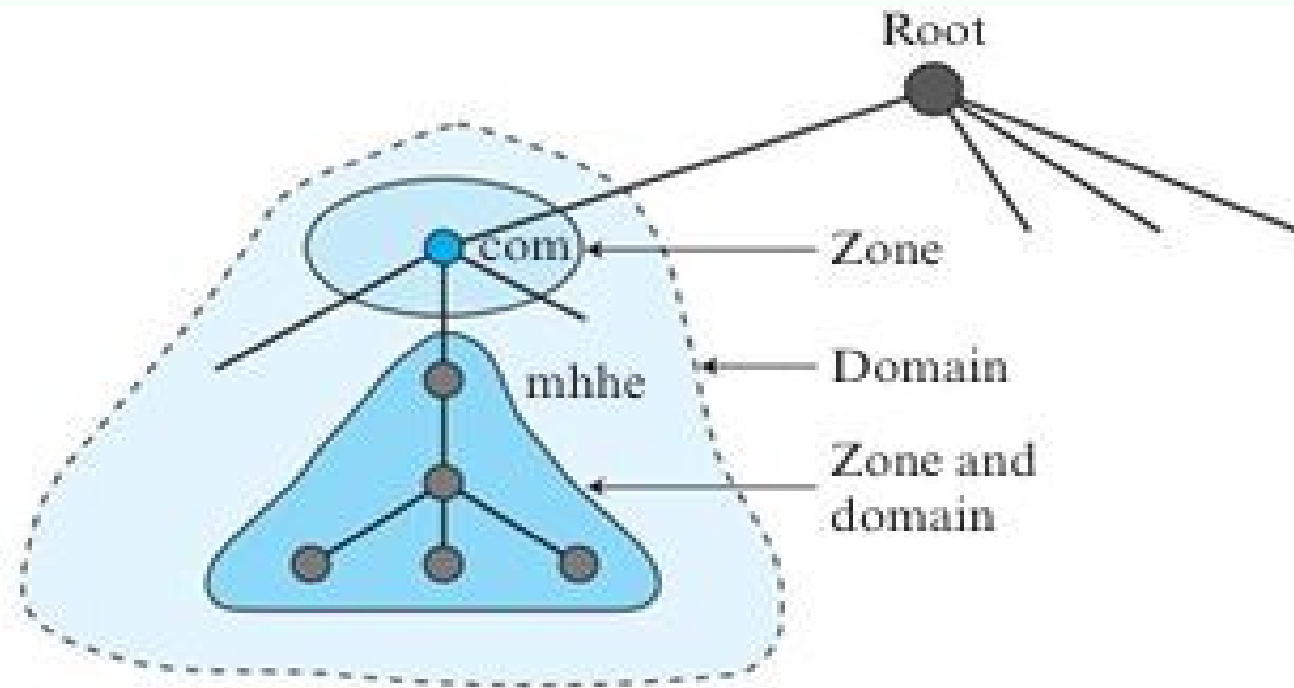
Hierarchy of name servers

Figure 2.39 *Hierarchy of name servers*



Zone

Figure 2.40 Zone



Root Server

- ▶ A Root Server is a DNS server whose zone covers the entire domain name space.
- ▶ Instead of storing detailed information about all domains, it delegates authority to lower-level DNS servers and maintains references to them. Multiple root servers are distributed across the world to improve reliability and availability of the DNS syst

Primary and Secondary Servers

- ▶ DNS uses two types of authoritative servers: Primary Servers and Secondary Servers.
- ▶ A Primary Server stores the original zone file and is responsible for creating, maintaining, and updating it.
- ▶ A Secondary Server obtains a copy of the zone file from a primary or another secondary server and stores it locally. Secondary servers cannot modify the zone file; any updates must be made on the primary server and then transferred to the secondary servers. Both primary and secondary servers are authoritative for their zones. The purpose of secondary servers is to provide redundancy and improve reliability, ensuring that DNS services continue even if the primary server fails. A server can act as a primary server for one zone and a secondary server for another zone.

DNS in the Internet

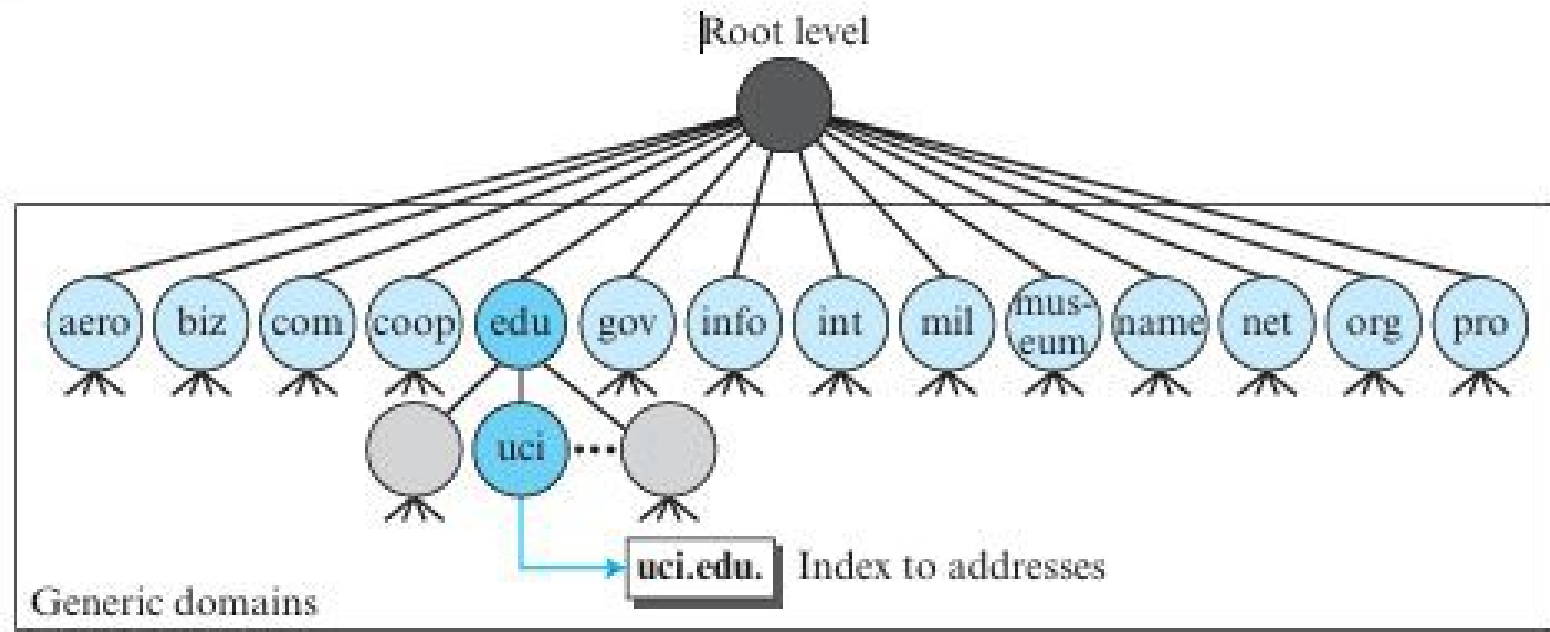
- ▶ DNS is a protocol used to translate domain names into IP addresses on the Internet.
- ▶ Originally, the Internet domain name space was divided into **Generic Domains, Country Domains, and Inverse Domains.**
- ▶ However, inverse domains, which were used to **find a hostname from an IP address, have been deprecated due to the rapid growth of the Internet.**
- ▶ Therefore, modern DNS mainly focuses on generic and country domains.

Generic Domains

- ▶ Generic Domains classify hosts based on their general purpose or type of organization. Each domain in the DNS hierarchy acts as an index to the domain name space database.
- ▶ Common generic domains include .com for commercial organizations, .edu for educational institutions, .org for non-profit organizations, .gov for government agencies, and .net for network service providers.
- ▶ These domains help organize and identify different types of Internet resources efficiently.

Genetic domains

Figure 2.41 *Generic domains*



Peer-to-Peer (P2P) Paradigm

- ▶ The Peer-to-Peer (P2P) paradigm is a network model in which computers communicate directly with each other without relying on a central server. Each computer, called a peer, can act as both a client and a server. The concept of P2P file sharing began with WWIVnet in 1987 and later evolved through systems such as Freenet (1999), Napster (1999), Gnutella (2000), Kazaa, BitTorrent, WinMX, and GUNet (2001).
- ▶ P2P networks became popular for file sharing because they enable decentralized communication and resource sharing among users.

P2P Networks

- ▶ A Peer-to-Peer (P2P) network is a distributed network in which Internet users share resources directly with one another. When a peer has a file, such as an audio, video, or document file, it makes the file available to other peers in the network. Any interested peer can connect directly to the computer storing the file and download it. After downloading, the peer can also share the same file with others, increasing the number of available copies in the network. As peers continuously join and leave the network, maintaining information about active peers and file locations becomes an important challenge. Based on how this information is managed, P2P networks are classified into **centralized and decentralized networks**.

Centralized P2P Networks

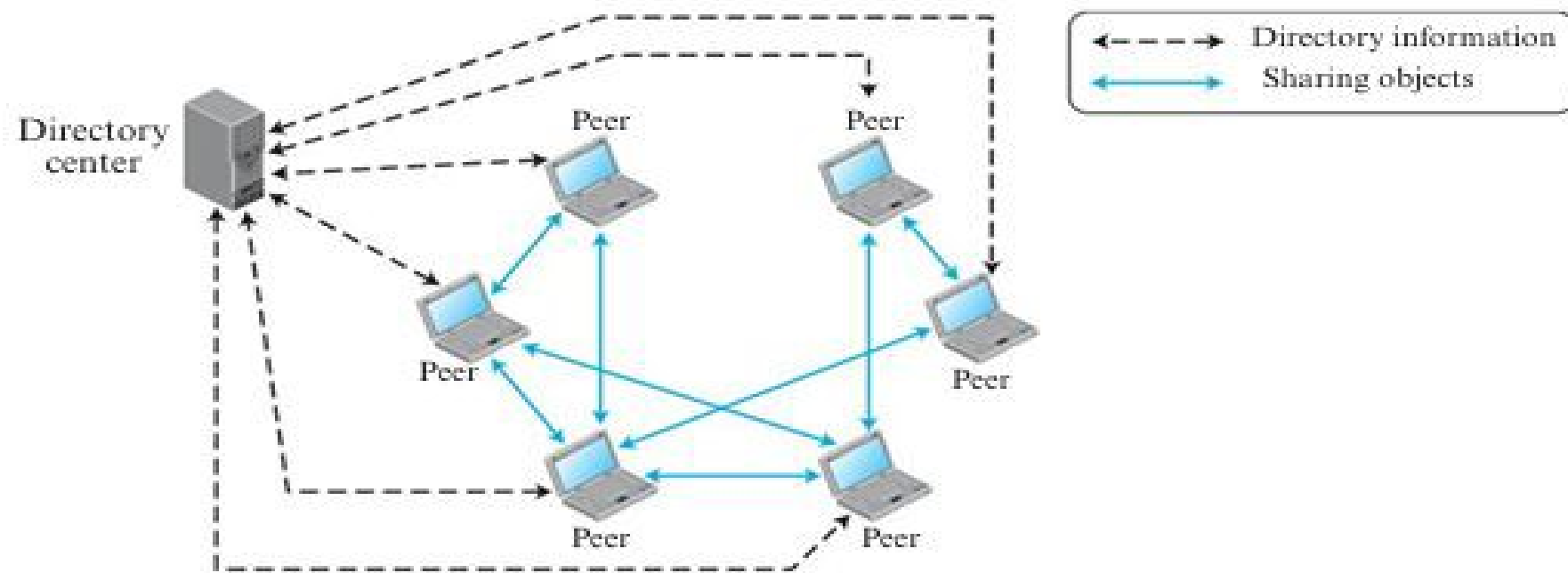
- ▶ A Centralized Peer-to-Peer (P2P) Network **combines the client-server and peer-to-peer paradigms and is therefore often called a hybrid P2P network**. In this model, a central server maintains a directory containing information about peers and the files they share, while the actual storage and transfer of files take place directly between peers. When a peer joins the network, it registers with the central server by providing its IP address and a list of files available for sharing. When another peer searches for a particular file, it sends a query to the central server. The server searches its directory and returns the IP addresses of peers that possess the requested file. The requesting peer then connects directly to one of those peers and downloads the file. The directory is continuously updated as peers join or leave the network.

Centralized P2P Networks

- ▶ Although centralized networks simplify file searching and directory management, they have several disadvantages.
- ▶ The central server can become a bottleneck due to heavy traffic, reducing system performance.
- ▶ It is also vulnerable to attacks and failures; if the central server becomes unavailable, the entire network may stop functioning.
- ▶ **Napster** was a well-known example of a centralized P2P network. Its reliance on central servers contributed to legal and operational challenges, eventually leading to its shutdown in 2001.

Centralized P2P Networks

Figure 2.46 Centralized network



Decentralized P2P Networks

- ▶ A Decentralized Peer-to-Peer (P2P) Network does not rely on a central server or directory system. **Instead, peers organize themselves into an overlay network, which is a logical network built on top of the physical network.** Based on how the peers are connected, decentralized networks are classified as unstructured or structured.
- ▶ In an **Unstructured P2P Network**, peers are connected randomly. To locate a file, a search query is **flooded across the network**, which generates high traffic and may not always find the desired file efficiently. Examples of unstructured P2P networks include Gnutella and Freenet.

Decentralized P2P Networks

- ▶ In a Structured P2P Network, peers are **connected according to predefined rules**, enabling faster and more efficient file searches. These networks commonly use a **Distributed Hash Table** (DHT) to organize and locate resources. DHT technology is widely used in systems such as distributed databases, content distribution systems, DNS, and P2P file-sharing applications.
- ▶ BitTorrent is a popular example of a structured P2P network that uses DHT for efficient resource discovery and file sharing.



BitTorrent

BitTorrent – A Popular Peer-to-Peer (P2P) Network

Introduction

- ▶ **BitTorrent**, designed by Bram Cohen, is one of the most widely used Peer-to-Peer (P2P) file-sharing protocols.
- ▶ Unlike traditional file-sharing systems, where a single peer provides the entire file, BitTorrent divides a large file into smaller pieces called **chunks**.
- ▶ Peers in the network simultaneously download and upload these chunks, making file distribution **faster and more efficient**.

Working of BitTorrent

- ▶ The group of peers participating in the sharing of a file is called a **swarm**.
- ▶ A peer that possesses the complete file is known as a **seed**, while a peer that has only some parts of the file and is still downloading the rest is called a **leech**.
- ▶ As peers download chunks, they also upload them to others, creating a collaborative sharing environment.

BitTorrent with a Tracker

- ▶ In the original BitTorrent design, a central entity called a **tracker** manages the swarm.
- ▶ When a new peer wants to download a file, it first obtains a **torrent file (metafile)** containing information about the file pieces and the tracker's address.
- ▶ The peer then contacts the tracker, which provides a list of neighboring peers participating in the torrent.
- ▶ The new peer downloads missing chunks from these peers and uploads available chunks to others. Once all chunks are collected, the peer becomes a seed and can continue helping other users.

Fairness and Efficiency Mechanisms

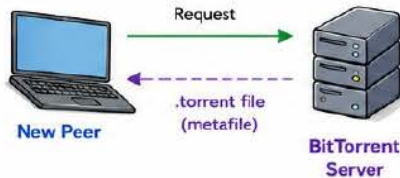
- ▶ BitTorrent uses several strategies to ensure fairness and efficient resource utilization.
- ▶ Each peer maintains connections with only a **limited number of neighbors, typically four**, to avoid overloading the network.
- ▶ Peers are categorized as **unchoked** (active connections) or **choked** (inactive connections).
- ▶ Every few seconds, peers evaluate neighboring nodes and prefer those offering better data transfer rates. BitTorrent also uses **optimistic unchoking**, which periodically allows a new peer to receive data even if it has not yet contributed. Another important strategy is **rarest-first**, where peers prioritize downloading the least available pieces first, ensuring balanced distribution of file chunks throughout the swarm.

BitTorrent: File Sharing Network works

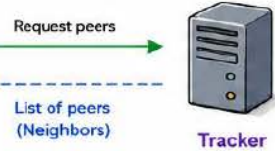
BitTorrent: How a P2P File Sharing Network Works

1 BitTorrent with a Tracker

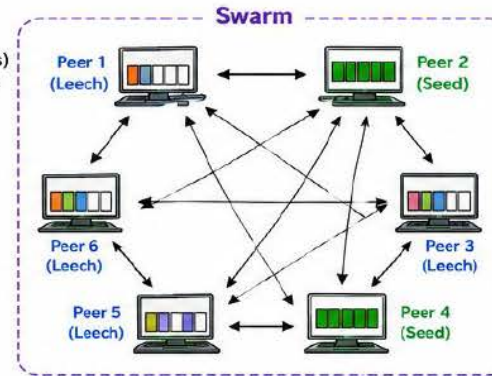
- 1 A user gets the torrent file from the BitTorrent server.



- 2 The peer contacts the tracker to get a list of peers.



- 3 The peer connects to other peers (neighbors) and exchanges pieces.



File (Content) Divided into 5 Pieces (Chunks)

1 2 3 4 5

Legend



Key Points

- File is divided into pieces (chunks).
- Peers download pieces from others and upload pieces they have.
- When a peer has all pieces, it becomes a Seed.
- The tracker helps new peers find others in the swarm.

Peer Connection Policy (for fairness and efficiency)



Tit-for-tat:
Upload to those who upload to you.



Choking / Unchoking:
Keep best performing peers unchoked.

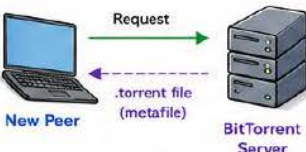


Optimistic Unchoke:
Every 30 sec, unchoke one random peer.

Rarest-First:
Download rarest pieces first.

2 Trackerless BitTorrent using DHT (Kademlia)

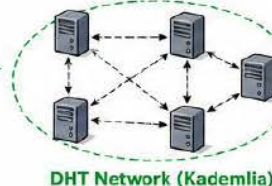
- 1 A peer gets the torrent file (metafile).



- 2 The peer computes the hash of the metadata (as the key).



- 3 The peer sends the key into the DHT (Kademlia network).



- 4 The DHT finds the node responsible for the key and returns the list of peers (Value).



- 5 The peer gets the list of peers and starts exchanging pieces (as in regular BitTorrent).



Benefits of Trackerless (DHT)

- ✓ No single point of failure
- ✓ Decentralized and scalable
- ✓ New peers can still join even if some nodes go down
- ✓ More resilient and efficient

BitTorrent is a collaborative P2P protocol that divides a file into pieces, and peers download and upload pieces simultaneously. It achieves fast, efficient and scalable file sharing. Modern versions use DHT to remove the need for a central tracker.



Trackerless BitTorrent

- ▶ One limitation of the original BitTorrent system was its dependence on the tracker. If the tracker failed, new peers could not join the swarm.
- ▶ To overcome this problem, modern BitTorrent implementations use a **Distributed Hash Table (DHT)**, particularly the **Kademlia DHT** algorithm. In this approach, the tracking function is distributed among multiple peers instead of relying on a central server. A joining peer sends a request to the DHT network, which locates the responsible node and returns a list of peers participating in the torrent. This eliminates the single point of failure and improves scalability and reliability.

Conclusion

- ▶ BitTorrent is an efficient and scalable P2P file-sharing protocol that distributes the workload among participating peers. By dividing files into chunks and encouraging peers to both upload and download data, BitTorrent achieves faster transfers and better resource utilization. The introduction of trackerless operation using DHT further enhanced its reliability by removing dependence on a central tracker.

Short Answer Questions (2 Marks)

1. Define a computer network.
2. List the advantages of computer networks.
3. What are end devices? Give examples.
4. What are connecting devices in a network?
5. Define transmission media.
6. Differentiate between wired and wireless media.
7. What is a PAN?
8. What is a LAN?
9. What is a MAN?
10. What is a WAN?
11. Define an intranet.
12. What is an internetwork?
13. What is the role of a router?
14. What is the role of a switch?

Short Answer Questions (2 Marks)

1. Define Point-to-Point WAN.
2. Define Switched WAN.
3. Differentiate between Internet and Intranet.
4. What is circuit switching?
5. What is packet switching?
6. Define ISP (Internet Service Provider).
7. What is DSL?
8. What is the TCP/IP protocol suite?
9. Why does TCP/IP use layering?
10. List the layers of the TCP/IP model.
11. What is encapsulation?
12. What is decapsulation?
13. Define multiplexing.
14. Define demultiplexing.
15. What is the client-server paradigm?
16. What is the peer-to-peer paradigm?

Short Essay Questions (5 Marks)

1. Explain the need for computer networks.
2. Describe the components of a computer network.
3. Explain different types of transmission media.
4. Compare LAN, MAN, and WAN.
5. Explain the concept of an internetwork.
6. Discuss the role of routers and switches in networking.
7. Explain Point-to-Point WAN and Switched WAN.
8. Differentiate between Internet and Intranet.
9. Explain circuit switching with an example.
10. Explain packet switching with an example.
11. Compare circuit-switched and packet-switched networks.
12. Describe the structure of the Internet.
13. Explain the different types of ISPs.
14. Discuss various methods of accessing the Internet.
15. Explain the TCP/IP protocol suite.
16. Discuss the advantages of layering in TCP/IP.
17. Explain the functions of the TCP/IP layers.
18. Describe encapsulation and decapsulation.
19. Explain multiplexing and demultiplexing.
20. Compare Client–Server and Peer-to-Peer paradigms.

Long Answer Questions (10 Marks)

1. Explain the components of a computer network with suitable diagrams.
2. Discuss different types of computer networks (PAN, LAN, CAN, MAN, WAN, GAN).
3. Explain the concept of internetworking and the role of routers and switches.
4. Describe Point-to-Point WAN and Switched WAN with examples.
5. Compare circuit switching and packet switching in detail.
6. Explain the structure of the Internet and the role of backbone, provider, and customer networks.
7. Discuss the functions and types of Internet Service Providers (ISPs).
8. Explain various methods used to access the Internet.
9. Describe the TCP/IP protocol suite and its layered architecture.
10. Explain the responsibilities of each layer in the TCP/IP model.
11. Explain logical communication in the TCP/IP protocol suite.
12. Discuss encapsulation and decapsulation with neat diagrams.
13. Explain multiplexing and demultiplexing with suitable examples.
14. Describe the client-server paradigm and its applications.
15. Explain the peer-to-peer paradigm, its advantages, disadvantages, and applications.

Important University-Level Questions

1. Explain the layered architecture of the TCP/IP protocol suite.
2. Compare circuit switching and packet switching.
3. Explain the structure of the Internet.
4. Discuss the role of routers and switches in internetworking.
5. Explain encapsulation and decapsulation in TCP/IP.
6. Describe multiplexing and demultiplexing.
7. Compare client-server and peer-to-peer paradigms.
8. Explain different methods of Internet access.
9. Discuss the various types of computer networks.
10. Explain the components of a computer network.